

Algoritmi e Strutture Dati – Prova di Laboratorio

23/07/2026

Istruzioni

Risolvere il seguente esercizio implementando un programma in un singolo file `.cpp`, completo di funzione *main*. Si presti particolare attenzione alla formattazione dell'input e dell'output, e alla complessità target indicata per ciascuna funzionalità. Nel caso la complessità target non sia specificata, si richiede che sia la migliore possibile. La lettura dell'input e la scrittura dell'output **DEVONO** essere effettuate tramite gli stream **cin** e **cout** rispettivamente. La correzione avverrà prima in maniera automatica inviando il file `.cpp` al server indicato in aula. Quest'ultimo esegue dei test confrontando l'output prodotto dalla vostra soluzione con l'output atteso. In caso la verifica abbia esito positivo sarà possibile consegnare il compito, il quale verrà valutato dai docenti in termini di complessità. Si ricorda che è possibile testare la correttezza del vostro programma in locale su un sottoinsieme dei input/output utilizzati nella seguente maniera. I file di input e output per i test sono nominati secondo lo schema: `input0.txt output0.txt input1.txt output1.txt ...`. Per effettuare le vostre prove potete utilizzare il comando del terminale per la redirectione dell'input. Ad esempio

```
./compilato < input0.txt
```

effettua il test del vostro codice sui dati contenuti nel primo file di input, assumendo che `compilato` contenga la compilazione della vostra soluzione e che si trovi nella vostra home directory. Dovete aspettarvi che l'output coincida con quello contenuto nel file `output0.txt`. Per effettuare un controllo automatico sul primo file input `input0.txt` potete eseguire la sequenza di comandi

```
./compilato < input0.txt | diff - output0.txt
```

Questa esegue la vostra soluzione e controlla le differenze fra l'output prodotto e quello corretto.

Esercizio

Si consideri un sistema per la gestione dei pazienti in attesa presso un pronto soccorso. Ogni paziente è caratterizzato da una stringa **codice**, che rappresenta il suo identificativo univoco; un intero **priorità**, $0 \leq \text{priorità} \leq P-1$, che rappresenta la sua priorità; un intero **istante**, contenente l'istante temporale di accesso. L'ordine di priorità è inverso rispetto al valore del campo **priorità**: valori numerici minori indicano priorità maggiori, 0 è la priorità massima.

Il sistema riceve nel tempo operazioni di inserimento e di estrazione.

Un paziente ha precedenza rispetto a un altro secondo i seguenti criteri: i) priorità dalla maggiore alla minore, considerando che valori numerici più piccoli rappresentano priorità maggiori; ii) a parità di priorità, istante di arrivo crescente; iii) a ulteriore parità, codice in ordine lessicografico crescente.

Le operazioni possibili sono le seguenti:

- **I**: Inserimento, che inserisce un nuovo paziente nel sistema;
- **E**: Estrazione, che estrae il paziente avente precedenza maggiore. Se non ci sono pazienti, l'operazione non ha alcun effetto.

Si assume che gli istanti associati alle operazioni siano non decrescenti nell'ordine in cui le operazioni compaiono nell'input. Nel caso di più operazioni con lo stesso istante, il loro ordine è quello con cui compaiono nell'input.

Si definisce tempo di attesa $t_{attesa} = \text{istante}_{estrazione} - \text{istante}_{inserimento}$.

Considerando tutti i pazienti estratti, si richiede di determinare, per ogni livello di priorità, il paziente avente il tempo massimo di attesa. A parità di t_{attesa} , scegliere il paziente estratto prima.

Si scriva un programma che:

- legga la sequenza delle operazioni; mantenga i pazienti in attesa all'interno di una struttura dati;
- esegua le operazioni di inserimento e di estrazione nell'ordine in cui compaiono nell'input;
- stampi, per ogni *priorità* il *codice* del paziente avente t_{attesa} massimo tra i pazienti estratti con quella priorità. A parità di t_{attesa} , scegliere il paziente estratto prima. Nel caso in cui non ci siano estrazioni per una *priorità*, stampi -1 .

L'**input** è formattato nel seguente modo: la prima riga contiene gli interi N e P rappresentanti rispettivamente il numero complessivo di operazioni e il numero di livelli di priorità. Seguono N righe, ciascuna contenente una delle seguenti operazioni: i) **I codice priorità istante** oppure ii) **E istante**. Si assume che i codici dei pazienti siano univoci.

L'**output** contiene gli elementi della soluzione, uno per riga.

Esempio

Input

```
10 3
I CSNVRL00R51F268C 1 4
I CRDLRT39D02C561M 2 5
E 6
I CRESTN60P06C818U 2 7
I FRNGLS91E53H223M 1 9
E 10
E 15
I RNCGU057B48C390U 0 17
I MRTLNA75P08G304Q 0 18
E 25
```

Output

```
RNCGU057B48C390U
CSNVRL00R51F268C
CRDLRT39D02C561M
```